

milestone5_renyi

April 4, 2026

1 Milestone 5 Submission

In this iteration of our MIA methods, we try out two new features, mod-renyi and max-renyi-k (Li et al., 2024) to see how our features perform in combination.

2 Imports and setup

```
[6]: import sys
import os
import pickle
import warnings
warnings.filterwarnings('ignore')
import torch

from google.colab import drive

drive.mount('/content/drive')
SAVE_DIR = '/content/drive/MyDrive/Colab Notebooks/VLM_MIA/'

sys.path.insert(0, SAVE_DIR)

print(f'sys.path after insert: {sys.path}')
if os.path.exists(SAVE_DIR):
    print(f'Contents of {SAVE_DIR}: {os.listdir(SAVE_DIR)}')
else:
    print(f'{SAVE_DIR} does not exist - creating it.')
    os.makedirs(SAVE_DIR, exist_ok=True)
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

sys.path after insert: ['/content/drive/MyDrive/Colab Notebooks/VLM_MIA/', '/content/drive/MyDrive/Colab Notebooks/VLM_MIA/', '/content', '/env/python', '/usr/lib/python312.zip', '/usr/lib/python3.12', '/usr/lib/python3.12/lib-dynload', '', '/usr/local/lib/python3.12/dist-packages', '/usr/lib/python3/dist-packages', '/usr/local/lib/python3.12/dist-packages/IPython/extensions', '/root/.ipython']

Contents of /content/drive/MyDrive/Colab Notebooks/VLM_MIA/: ['__pycache__',

```
'milestone5_renyi.ipynb', 'batches.py', 'classifier.py', 'config.py',  
'models.py', 'features.py', 'train.csv']
```

```
[ ]: from models import load_data, load_model_and_processor,   
      ↪ load_base_model_and_processor  
from batches import process_dataset_in_batches  
from classifier import build_membership_classifier, generate_submission  
  
os.makedirs(SAVE_DIR, exist_ok=True)  
  
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')  
print(f'Device: {device}')  
  
# Always load splits - needed for labels and test IDs even when using cached  
↪ features.  
splits = load_data()
```

Device: cuda

Loading dataset: UBC-SLIME/colx585_group_project_data

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

train.parquet: 0%| | 0.00/334M [00:00<?, ?B/s]

dev.parquet: 0%| | 0.00/67.0M [00:00<?, ?B/s]

test.parquet: 0%| | 0.00/338M [00:00<?, ?B/s]

Generating train split: 0%| | 0/6000 [00:00<?, ? examples/s]

Generating validation split: 0%| | 0/1200 [00:00<?, ? examples/s]

Generating test split: 0%| | 0/6000 [00:00<?, ? examples/s]

Train: 6000 samples (1500 members)

Validation: 1200 samples (300 members)

Test: 6000 samples

3 Extract Features

```
[ ]: !pip install num2words
```

Collecting num2words

Downloading num2words-0.5.14-py3-none-any.whl.metadata (13 kB)

Collecting docopt>=0.6.2 (from num2words)

Downloading docopt-0.6.2.tar.gz (25 kB)

Preparing metadata (setup.py) ... done

Downloading num2words-0.5.14-py3-none-any.whl (163 kB)

163.5/163.5 kB

6.6 MB/s eta 0:00:00

Building wheels for collected packages: docopt

Building wheel for docopt (setup.py) ... done

Created wheel for docopt: filename=docopt-0.6.2-py2.py3-none-any.whl
size=13706

sha256=112b4055be9a41a6afc9278cc8f23bb62f87484a081f781838b72633492361bd

Stored in directory: /root/.cache/pip/wheels/1a/bf/a1/4cee4f7678c68c5875ca89ea
ccf460593539805c3906722228

Successfully built docopt

Installing collected packages: docopt, num2words

Successfully installed docopt-0.6.2 num2words-0.5.14

```
[ ]: # Load fine-tuned (target) and base (reference) models
processor, model = load_model_and_processor(device)
base_processor, base_model = load_base_model_and_processor(device)
```

Using dtype: torch.bfloat16

processor_config.json: 0.00B [00:00, ?B/s]

chat_template.jinja: 0%| | 0.00/403 [00:00<?, ?B/s]

config.json: 0.00B [00:00, ?B/s]

tokenizer_config.json: 0%| | 0.00/838 [00:00<?, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

Loading model: UBC-SLIME/colx_585_vlm

model.safetensors: 0%| | 0.00/513M [00:00<?, ?B/s]

Loading weights: 0%| | 0/471 [00:00<?, ?it/s]

generation_config.json: 0%| | 0.00/135 [00:00<?, ?B/s]

Model loaded.

Using dtype: torch.bfloat16

processor_config.json: 0%| | 0.00/68.0 [00:00<?, ?B/s]

chat_template.json: 0%| | 0.00/424 [00:00<?, ?B/s]

preprocessor_config.json: 0%| | 0.00/486 [00:00<?, ?B/s]

config.json: 0.00B [00:00, ?B/s]

tokenizer_config.json: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

added_tokens.json: 0%| | 0.00/92.0 [00:00<?, ?B/s]

special_tokens_map.json: 0.00B [00:00, ?B/s]

Loading model: HuggingFaceTB/SmolVLM-Base

You are using a model of type idefics3 to instantiate a model of type smolvlm. This is not supported for all configurations of models and can yield errors.

```
model.safetensors:  0%|          | 0.00/4.49G [00:00<?, ?B/s]
Loading weights:    0%|          | 0/657 [00:00<?, ?it/s]
generation_config.json:  0%|          | 0.00/132 [00:00<?, ?B/s]
Model loaded.
```

```
[ ]: train_features = process_dataset_in_batches(
    model, processor, splits['train'], has_labels=True, batch_size=200,
    base_model=base_model, base_processor=base_processor
)
train_features.to_csv(f'{SAVE_DIR}train.csv', index=False)
print('Train features saved.')
```

Processing 6000 samples in batches of 200...

```
Batch 1/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:27<00:00, 1.34s/it]
Batch 2/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:31<00:00, 1.36s/it]
Batch 3/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:33<00:00, 1.37s/it]
Batch 4/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:34<00:00, 1.37s/it]
Batch 5/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:34<00:00, 1.37s/it]
Batch 6/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:32<00:00, 1.36s/it]
Batch 7/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:33<00:00, 1.37s/it]
Batch 8/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:36<00:00, 1.38s/it]
Batch 9/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:33<00:00, 1.37s/it]
Batch 10/30 (200 samples)
Extracting features: 100%|          | 200/200 [04:33<00:00, 1.37s/it]
Batch 11/30 (200 samples)
```

Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]
 Batch 12/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 13/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 14/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 15/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 16/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 17/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]
 Batch 18/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 19/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 20/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]
 Batch 21/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 22/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 23/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 24/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:33<00:00, 1.37s/it]
 Batch 25/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 26/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:33<00:00, 1.37s/it]
 Batch 27/30 (200 samples)

```

Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
    Batch 28/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
    Batch 29/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:30<00:00, 1.35s/it]
    Batch 30/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:30<00:00, 1.35s/it]
Done. 6000 samples processed.
Train features saved.

```

```

[7]: val_features = process_dataset_in_batches(
    model, processor, splits['validation'], has_labels=True, batch_size=200,
    base_model=base_model, base_processor=base_processor
)
val_features.to_csv(f'{SAVE_DIR}val.csv', index=False)
print('Val features saved.')

```

```

Processing 1200 samples in batches of 200...
    Batch 1/6 (200 samples)
Extracting features: 100%|      | 200/200 [04:30<00:00, 1.35s/it]
    Batch 2/6 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
    Batch 3/6 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
    Batch 4/6 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
    Batch 5/6 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
    Batch 6/6 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
Done. 1200 samples processed.
Val features saved.

```

```

[8]: test_features = process_dataset_in_batches(
    model, processor, splits['test'], has_labels=False, batch_size=200,
    base_model=base_model, base_processor=base_processor
)

```

```
test_features.to_csv(f'{SAVE_DIR}test.csv', index=False)
print('Test features saved.')
```

Processing 6000 samples in batches of 200...

```
Batch 1/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 2/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 3/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 4/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
Batch 5/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
Batch 6/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 7/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
Batch 8/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
Batch 9/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 10/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:32<00:00, 1.36s/it]
Batch 11/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 12/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:33<00:00, 1.37s/it]
Batch 13/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:33<00:00, 1.37s/it]
Batch 14/30 (200 samples)
Extracting features: 100%|      | 200/200 [04:31<00:00, 1.36s/it]
Batch 15/30 (200 samples)
```

Extracting features: 100%| | 200/200 [04:33<00:00, 1.37s/it]
 Batch 16/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:33<00:00, 1.37s/it]
 Batch 17/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]
 Batch 18/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 19/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 20/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:29<00:00, 1.35s/it]
 Batch 21/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 22/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 23/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]
 Batch 24/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 25/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 26/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]
 Batch 27/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 28/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:30<00:00, 1.35s/it]
 Batch 29/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:31<00:00, 1.36s/it]
 Batch 30/30 (200 samples)
 Extracting features: 100%| | 200/200 [04:32<00:00, 1.36s/it]

Done. 6000 samples processed.
Test features saved.

4 Load features (new session only)

```
[ ]: # load this instead of section 1 if starting new session
# import pandas as pd
# train_features = pd.read_csv(f'{SAVE_DIR}train.csv')
# val_features = pd.read_csv(f'{SAVE_DIR}val.csv')
# test_features = pd.read_csv(f'{SAVE_DIR}test.csv')
# print('Features loaded.')
# print(f' train: {train_features.shape} val: {val_features.shape} test:
↳ {test_features.shape}')
# print(f' Columns: {list(train_features.columns)}')
```

5 Train classifiers

In this section, we check to see which features have the highest AUC scores. We compare the classifier methods logistic regression, XGBoost, ensemble, random forest, gradient boost, and MLP, and find the ensemble method to have the highest score on the validation set.

```
[9]: results = build_membership_classifier(train_features, val_features)

best_name = max(
    (n for n in results if n not in {"Loss", "Zlib Ratio", "Min-K% (k=10%)",
↳ "Min-K%+ (k=10%)"}),
    key=lambda x: results[x]['auc']
)
best = results[best_name]
print(f'\nSelected for submission: {best_name} AUC={best["auc"]:.4f}')
```

Features: 59 | Train: 6000 | Val: 1200

Individual feature AUCs (top 10):

caption_loss_diff	AUC=0.7301
perplexity_ratio	AUC=0.7301
temp_sensitivity_ext_diff	AUC=0.7009
temp_sensitivity_diff	AUC=0.6904
ref_min_k10_prob	AUC=0.6872
ref_loss_at_T05	AUC=0.6863
ref_caption_loss	AUC=0.6812
ref_caption_perplexity	AUC=0.6812
ref_temp_sensitivity	AUC=0.6804
loss_ratio	AUC=0.6796

Training Logistic Regression...

AUC=0.7864 TPR@FPR=0.1=0.2967

Training XGBoost...

AUC=0.7832 TPR@FPR=0.1=0.2967

Top-5: ['perplexity_ratio', 'caption_loss_diff', 'ref_temp_sensitivity', 'ref_min_k10_prob', 'temp_sensitivity_ext_diff']

Building ensemble (simple average)...

AUC=0.7879 TPR@FPR=0.1=0.2867

Summary

Logistic Regression	AUC=0.7864	TPR@FPR=0.1=0.2967
XGBoost	AUC=0.7832	TPR@FPR=0.1=0.2967
Ensemble	AUC=0.7879	TPR@FPR=0.1=0.2867

Best: Ensemble AUC=0.7879

Selected for submission: Ensemble AUC=0.7879

Checking to see what features are contributing most (curious to see how much of a difference our new features, max and mod renyi-k, are making).

```
[10]: from sklearn.metrics import roc_auc_score
import numpy as np

feat_cols = results[best_name]["feat_cols"]
medians = results[best_name]["medians"]

print("All feature AUCs (sorted):")
aucs = []
for col in feat_cols:
    try:
        auc = roc_auc_score(train_features["label"], train_features[col])
        aucs.append((col, max(auc, 1 - auc)))
    except:
        pass
aucs.sort(key=lambda x: -x[1])
for col, auc in aucs:
    print(f" {col:40s} AUC={auc:.4f}")
```

All feature AUCs (sorted):

caption_loss_diff	AUC=0.7301
perplexity_ratio	AUC=0.7301
temp_sensitivity_ext_diff	AUC=0.7009
temp_sensitivity_diff	AUC=0.6904
ref_min_k10_prob	AUC=0.6872
ref_loss_at_T05	AUC=0.6863
ref_caption_loss	AUC=0.6812
ref_caption_perplexity	AUC=0.6812
ref_temp_sensitivity	AUC=0.6804

loss_ratio	AUC=0.6796
ref_min_k20_prob	AUC=0.6734
ref_loss_at_T025	AUC=0.6719
ref_temp_sensitivity_ext	AUC=0.6698
ref_min_token_prob	AUC=0.6617
ref_loss_at_T20	AUC=0.6470
ref_loss_at_T40	AUC=0.6412
mean_prob_diff	AUC=0.6175
mean_entropy_diff	AUC=0.6112
mod_renyi_a20_diff	AUC=0.6106
ref_mean_token_prob	AUC=0.6043
ref_mod_renyi_a20	AUC=0.5976
ref_mean_entropy	AUC=0.5935
min_k10_prob_diff	AUC=0.5931
ref_mod_renyi_a05	AUC=0.5833
ref_max_renyi_k20_a20	AUC=0.5779
max_renyi_k20_a20_diff	AUC=0.5769
mod_renyi_a05_diff	AUC=0.5769
min_k20_prob_diff	AUC=0.5699
max_renyi_k20_a05_diff	AUC=0.5526
ref_max_renyi_k20_a05	AUC=0.5520
ref_max_renyi_k10_a20	AUC=0.5518
max_renyi_k10_a20_diff	AUC=0.5467
ref_std_token_prob	AUC=0.5242
ref_max_renyi_k10_a05	AUC=0.5180
max_renyi_k10_a05_diff	AUC=0.5172
temp_sensitivity	AUC=0.5061
min_token_prob	AUC=0.5053
zlib_ratio	AUC=0.5042
min_k20_prob	AUC=0.5038
max_renyi_k10_a05	AUC=0.5036
temp_sensitivity_ext	AUC=0.5034
loss_at_T05	AUC=0.5034
mean_entropy	AUC=0.5032
max_renyi_k10_a20	AUC=0.5032
mod_renyi_a05	AUC=0.5032
zlib_entropy	AUC=0.5031
min_k10_prob	AUC=0.5030
loss_at_T025	AUC=0.5030
caption_loss	AUC=0.5024
caption_perplexity	AUC=0.5024
mod_renyi_a20	AUC=0.5022
mean_token_prob	AUC=0.5020
max_renyi_k20_a20	AUC=0.5019
caption_length	AUC=0.5018
ref_caption_length	AUC=0.5018
max_renyi_k20_a05	AUC=0.5014
loss_at_T40	AUC=0.5008

std_token_prob	AUC=0.5006
loss_at_T20	AUC=0.5002

Checking the above classifier methods against a few others to see if we can get better results (Claude helped me with this logic):

```
[11]: from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.preprocessing import StandardScaler
import numpy as np

feat_cols = results[best_name]["feat_cols"]
medians = results[best_name]["medians"]

# Reuse cleaned data
X_train = train_features[feat_cols].replace([np.inf, -np.inf], np.nan).
    ↪ fillna(medians).fillna(0)
X_val = val_features[feat_cols].replace([np.inf, -np.inf], np.nan).
    ↪ fillna(medians).fillna(0)
y_train = train_features["label"]
y_val = val_features["label"]

extra_models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=500, max_depth=6, min_samples_leaf=5,
        class_weight="balanced", random_state=42, n_jobs=-1,
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=500, learning_rate=0.01, max_depth=3,
        subsample=0.8, random_state=42,
    ),
    "MLP": MLPClassifier(
        hidden_layer_sizes=(64, 32), max_iter=1000,
        early_stopping=True, random_state=42,
    ),
}

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc = scaler.transform(X_val)

print(" Additional Classifiers ")
for name, clf in extra_models.items():
    # MLP needs scaled input, tree models don't but it won't hurt
    if name == "MLP":
        clf.fit(X_train_sc, y_train)
        proba = clf.predict_proba(X_val_sc)[: , 1]
```

```

else:
    clf.fit(X_train, y_train)
    proba = clf.predict_proba(X_val.values)[:, 1]

auc = roc_auc_score(y_val, proba)
fpr, tpr, _ = roc_curve(y_val, proba)
tpr_at_01 = tpr[np.argmin(np.abs(fpr - 0.1))]
print(f"   {name:25s}   AUC={auc:.4f}   TPR@FPR=0.1={tpr_at_01:.4f}")

```

Additional Classifiers

Random Forest	AUC=0.7798	TPR@FPR=0.1=0.2767
Gradient Boosting	AUC=0.7848	TPR@FPR=0.1=0.3000
MLP	AUC=0.7620	TPR@FPR=0.1=0.3067

Let's keep the best one, the ensemble method.

```

[12]: classifier_path = f'{SAVE_DIR}classifier.pkl'
with open(classifier_path, 'wb') as f:
    pickle.dump(best, f)
print(f'Classifier saved to {classifier_path}')

submission = generate_submission(
    splits['test'], test_features, best,
    output_path='submission.csv'
)
print(submission.head())

```

Classifier saved to /content/drive/MyDrive/Colab
Notebooks/VLM_MIA/classifier.pkl
Submission saved to submission.csv (6000 rows)
Score range: [0.0189, 0.8826]

	id	is_member
0	cauldron_ln_8044	0.502428
1	cauldron_ln_7863	0.663584
2	mix_SI_UT_5400_19	0.133493
3	mix_SI_UT_5400_896	0.386312
4	cauldron_ln_17842	0.117368

```

[14]: from google.colab import files
files.download('submission.csv')

```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>